

The Myth of Preemption

A Critique of Preemptive RTOS in Single-Core Systems

Marcos Roberto Braga

Informal Institute of Education, Science and Technology

noelmr_b_tec@yahoo.com

Pedro Henrique Cerqueira Braga

Collaborator

September 12, 2026

Abstract

This work deconstructs what we call the *Myth of Preemption*: the widespread belief that real-time systems require preemptive RTOS, and that preemption in single-core constitutes real parallelism. We argue that (i) preemption in single-core is time-slicing, not parallelism; (ii) preemption introduces problems (race conditions, deadlocks, priority inversion, timing jitter) that would not exist in a deterministic cooperative model; (iii) for *hard real-time*, determinism is more important than abrupt reactivity; and (iv) the industry systematically confuses the two regimes. We propose CDC-P (Concurrency Deterministic Control with Preemption), a cooperative kernel that implements real preemption through queue reordering, without abrupt task interruption. Empirical results on five architectures (PIC16, PIC18, AVR, RISC-V, ARM Cortex-M) demonstrate that the proposed model is more deterministic, more resource-efficient, and safer than the classic preemptive model.

Keywords: RTOS, preemption, myth, determinism, real-time systems, cooperative, micro-controllers.

1 Introduction

The embedded systems industry has adopted, over the last two decades, a widespread belief: *systems with multiple tasks require preemptive RTOS*. This belief is rarely questioned, and its adoption is often justified by arguments from authority (“*everyone uses it*”) or by circular fallacies (“*complex systems require complexity*”).

This work questions that belief on three fronts:

1. **Technical front:** we demonstrate that preemption in single-core is not real parallelism, but time-slicing, and that this distinction is systematically ignored by the literature and the industry.
2. **Conceptual front:** we argue that preemption introduces problems (race conditions, deadlocks, jitter) that would not exist in a deterministic cooperative model, and that these problems require complex solutions (mutexes, priority inheritance, critical sections) that preemption itself made necessary.
3. **Practical front:** we propose CDC-P (Concurrency Deterministic Control with Preemption), a cooperative kernel that implements real preemption through queue reordering, and we validate it empirically on five microcontroller architectures.

1.1 The Emergency Room Analogy

Consider an emergency room with a single doctor and five patients. Patients arrive with different urgencies. The industry-standard practice (preemptive RTOS) would correspond to:

“The doctor begins operating on patient A. In the middle of surgery, patient B arrives and the doctor interrupts A’s operation, saves the state (chest open), and goes to attend B. In the middle of B’s surgery, patient C arrives. The doctor interrupts B, saves the state, and goes to attend C. And so on. Neither patient ends up well.”

The correct practice (CDC-P) would correspond to:

*“The doctor evaluates all patients. Decides which is most critical. Operates on the most critical **completely**. Then operates on the next. And so on. All patients end up well.”*

The analogy exposes the absurdity of abrupt preemption: **no one would operate on two hearts at the same time**. But that is exactly what FreeRTOS does when it preempts a task in the middle of execution.

1.2 Contributions

This work presents the following contributions:

- An epistemological critique of preemption in single-core;
- A technical analysis of the problems introduced by preemption;
- A proposal for a deterministic cooperative model with preemption through queue reordering;
- Empirical validation on five MCU architectures;
- Comparative data with the classic preemptive model.

2 Theoretical Background

2.1 Liu & Layland (1973)

The foundational paper by Liu and Layland [1] established the theoretical foundations of preemptive real-time scheduling. Its premises are:

- Deadline equal to period;
- Independent tasks;
- Absence of shared resources;
- Absence of race conditions;
- Absence of overhead.

In practice, **none of these premises holds**. Tasks share resources, race conditions occur, overhead exists. Therefore, the mathematical guarantee of Liu and Layland is theoretical, not real.

2.2 Dijkstra (1968, 1972)

Dijkstra [2,3] established two fundamental principles:

1. “*Go To Statement Considered Harmful*” (1968): the programmer must be able to reason about the state of the program at any point.
2. “*Simplicity is prerequisite for reliability*” (1972): simplicity is a prerequisite for reliability.

We argue that preemption violates the first principle (it is a disguised *goto*: it saves the PC in one place, restores it from another, and switches the flow of control beneath the code) and ignores the second (it introduces complexity that would not exist in a cooperative model).

2.3 Rate Monotonic Scheduling and Its Limitations

Rate Monotonic Scheduling (RMS) [1] establishes that the maximum processor utilization is given by:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1) \quad (1)$$

where C_i is the execution time and T_i is the period of task i . For $n \rightarrow \infty$, $U \rightarrow \ln 2 \approx 0.693$.

However, this formula **ignores context-switch overhead**. In a system with many tasks and short deadlines, the overhead can consume 10–30% of the CPU, invalidating the theoretical guarantee.

3 The Contradiction of Preemption in Single-Core

3.1 Definition of Real-Time

Real-time **does not mean “fast”**. It means: *the system responds within a predictable deadline*. There are two types:

- **Hard Real-Time**: missing a deadline is a catastrophic failure (airbag, pacemaker);
- **Soft Real-Time**: missing a deadline is an acceptable degradation (streaming, games).

For **hard real-time**, what matters is **determinism**, not speed.

3.2 The Industry’s Standard Argument

The industry states:

“Use preemption when there are rigid real-time requirements.”

This argument is **contradictory**:

1. Preemption introduces **jitter** (unpredictable latency);
2. Jitter **breaks** mathematical guarantees;
3. Therefore, preemption is **inadequate** for hard real-time.

The cooperative model, on the other hand, is **deterministic**:

- No scheduling jitter;

- No race conditions;
- No deadlocks;
- No priority inversion.

For hard real-time, cooperative is better.

3.3 The Fiction of Parallelism

In single-core, there is no real parallelism. What exists is:

- **Context switch:** save registers, load new ones;
- **Time slice:** task A runs 10ms, B runs 10ms;
- **Interrupt:** timer interrupts A, kernel decides to run B.

This is **not parallelism**. It is **make-believe**. It is a useful fiction for selling tools, but dangerous in production.

4 Problems Introduced by Preemption

4.1 Race Conditions

Since tasks are not atomic, any access to shared data can be corrupted. This requires **mutexes**, which introduce:

- **Priority inversion:** low-priority task holds mutex;
- **Deadlocks:** two tasks waiting for each other's resources;
- **Starvation:** task never obtains the resource.

4.2 Priority Inversion

The classic case is the **Mars Pathfinder** (1997), where a low-priority task blocked a high-priority one, causing repeated resets. The solution required *priority inheritance* in VxWorks.

4.3 Timing Jitter

Each context switch introduces 5–15 μ s of latency (Cortex-M). At 1000 switches/s, this consumes 10ms/s. More seriously: the accumulated **jitter** makes the system clock drift.

4.4 Impeded Clocks

Systems with FreeRTOS frequently exhibit **freezing** and **jumping** in second-display clocks. This occurs because the clock task depends on `vTaskDelay`, which:

- Rounds to tick multiples (jitter of 1–10ms);
- Depends on scheduling (latency of 1–10ms);
- Can be blocked by mutex (variable latency).

External RTC does not solve this, because reading the RTC is also subject to the same problems.

5 The CDC-P Model

5.1 Overview

CDC-P (Concurrency Deterministic Control with Preemption) is a cooperative kernel that implements real preemption through **queue reordering**, without abrupt task interruption. Its main characteristics are:

- **Atomic tasks:** each task is a function that always returns;
- **Scheduling by modular arithmetic:** $(\text{tick} - \text{timer_task}) \geq \text{period}$;
- **Three-dimensional priority:** spatial (order), temporal (period), situational (URG-S);
- **7 protection layers:** self-regulation, deadline blocking, Sheriff, Ombudsman, fail-safe, CDC-P, URG-S;
- **No context switch:** shared stack.

5.2 Preemption by Reordering

CDC-P implements two preemption mechanisms:

5.2.1 URG-S (Situational Urgency)

An external event sets a flag. The dispatcher checks the flag **before everything else**. The urgent task executes **on the next scan**. Latency: scan time (25–112 μ s). Afterwards, the flag is cleared automatically.

5.2.2 CDC-P (Persistent Acceleration)

The application calls `CDC_EnablePreempt`. The task's period is reduced to 1 tick. The task executes on every iteration until `CDC_DisablePreempt`.

5.3 Emergency Room Analogy

URG-S corresponds to: *“Critical patient arrives; doctor finishes the current consultation and attends the critical one next.”*

CDC-P corresponds to: *“Hospital enters epidemic mode; all resources dedicate themselves to critical cases while the epidemic lasts.”*

In both cases, real preemption occurs without abrupt interruption.

6 Empirical Validation

6.1 Tested Platforms

CDC-P was implemented on five architectures:

6.2 Stress Tests

Under 1000 events/s (URG-S + CDC-P), the system remained stable:

Table 1: Validation platforms

MCU	Architecture	Clock	RAM	Flash
PIC16F628A	8 bits	4 MHz	224 B	2 KB
PIC18F6722	8 bits	8 MHz	4 KB	128 KB
AT90S8515	AVR 8 bits	8 MHz	512 B	8 KB
CH32V003F4P6	RISC-V	48 MHz	2 KB	16 KB
STM32F103	ARM Cortex-M3	72 MHz	20 KB	64 KB

Table 2: Stress test results (1000 Hz)

Metric	FreeRTOS (est.)	CDC-P (measured)
Cost per signaling	5–15 μ s	0.02 μ s
CPU load at 1000 Hz	0.5–1.5%	0.002%
Context switch	Mandatory	Nonexistent
URG-S latency	1–2 ticks	Scan (μ s)
CDC-P latency	1–2 ticks	1 tick
Stack per task	200–500 B	0 B
Kernel overhead	100–200 μ s	25 μ s
Determinism	Degrades	Constant
Clock precision	Drifts	Exact

6.3 Resources on PIC16F628A

7 Discussion

7.1 Determinism vs. Speed

The results demonstrate that CDC-P is **more deterministic** than FreeRTOS, maintaining **clock precision** even under load. This confirms the thesis that, for hard real-time, determinism is more important than raw speed.

7.2 Resources vs. Complexity

CDC-P uses a fraction of FreeRTOS’s resources, maintaining the same capabilities (preemption, priority, semaphores). This confirms the thesis that FreeRTOS’s complexity is self-referential: each mechanism exists to solve a problem created by the previous mechanism.

7.3 Portability

CDC-P was ported to five architectures without changes to the kernel. This confirms the thesis that the cooperative model is naturally portable, unlike the preemptive model (which requires a specific port for each architecture).

7.4 The Fiction of Preemption

Preemption in single-core is a **useful fiction**. It is useful for selling tools, but dangerous in production. In multi-core, preemption is **real**: two cores, two flows, true parallelism.

The industry should distinguish the two cases.

Table 3: Resource usage on PIC16F628A

Resource	Used	Free	Total
ROM	1360 words (66%)	688 words (34%)	2048 words
RAM	80 bytes (36%)	144 bytes (64%)	224 bytes
Stack	4 levels (50%)	4 levels (50%)	8 levels

8 Conclusion

This work presented a conceptual critique of the indiscriminate use of preemptive RTOS in single-core microcontrollers. We argued that:

1. Preemption in single-core is a fiction: there is no real parallelism;
2. This fiction introduces problems (race, deadlock, jitter) that would not exist in a cooperative model;
3. The cooperative model is more deterministic and more suitable for hard real-time;
4. CDC-P demonstrates that real preemption can be implemented through queue reordering, without abrupt interruption;
5. Empirical results on five architectures confirm the theses.

The industry needs to revisit its assumptions. Not every system requires FreeRTOS. In many cases, the deterministic cooperative model is more adequate, more efficient, and safer.

“Real preemption is only possible in multi-core. What preemptive RTOS do in single-core is a dangerous make-believe.”

Acknowledgments

The author thanks Pedro Henrique Cerqueira Braga for his collaboration in the development and testing of CDC-P.

References

- [1] LIU, C. L.; LAYLAND, J. W. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, v. 20, n. 1, p. 46–61, 1973.
- [2] DIJKSTRA, E. W. Go To Statement Considered Harmful. *Communications of the ACM*, v. 11, n. 3, p. 147–148, 1968.
- [3] DIJKSTRA, E. W. The Humble Programmer. *Communications of the ACM*, v. 15, n. 10, p. 859–866, 1972.
- [4] LABROSSE, J. J. *μC/OS-III: The Real-Time Kernel*. Micrium, 2010.
- [5] FreeRTOS Documentation. <https://www.freertos.org/>. Accessed: 2026.
- [6] BARR, M. *Programming Embedded Systems in C and C++*. O’Reilly, 1999.
- [7] JONES, M. What Really Happened on Mars? *NASA JPL*, 1997.

- [8] HYMEL, S. When to Use an RTOS. *Digi-Key Blog*, 2023.
- [9] HARVIE, L. The Uncomfortable Truth About RTOS in Embedded Systems. *LinkedIn*, 2024.
- [10] ASTIER, J. Y. Multiple FSMs vs RTOS. *LinkedIn*, 2024.